

NATIONAL ECONOMICS UNIVERSITY
COLLEGE OF TECHNOLOGY

OFFICIAL MIDTERM EXAMINATION

Academic Year: 2026–2027

FACULTY OF DATA SCIENCE & ARTIFICIAL INTELLIGENCE (FDA)

Semester: Fall 2026 Course: DSAI1003

FUNDAMENTAL PROGRAMMING CONCEPTS IN PYTHON

PROGRESS MIDTERM EXAM (40% QUIZ – 60% CODING PROBLEMS)

Paper Code: 101 | Duration: 60 Minutes | Difficulty: 80% Basic, 20% Medium

Candidate Full Name:

Student ID (MSSV): ...

Class / Cohort:

Exam Room / Seat No.:

Section	Sec A: Quiz (40%)	Sec B: Fill-in (30%)	Sec C: Code (30%)	Total (100%)	Examiner Signature
Max Score	40 pts	30 pts	30 pts	100 pts	
Score					

Instructions: Closed-book examination. All electronic devices and calculators are prohibited. Total score: 100 points (40% Quiz, 30% Fill-in, 30% Full Code; 80% Basic, 20% Medium). Mark MCQs directly in the answer sheet grid. Write code solutions in designated boxes. Do not detach pages.

MULTIPLE-CHOICE ANSWER SHEET (Write letters A, B, C, or D into corresponding boxes):

Q	1	2	3	4	5	6	7	8	9	10
Ans										
Q	11	12	13	14	15	16	17	18	19	20
Ans										

Section A: Multiple-Choice Quiz

(40 points — 20 questions, 2 pts each)

Select the single best answer. Questions marked [Basic] test core concepts; [Medium] test composite logic.

Q1. [Basic] How does the standard CPython implementation execute a Python source file (.py)?

- A. Directly compiles Python code into native x86/ARM machine code binaries prior to execution.
- B. Compiles source into bytecode (.pyc), which is executed by the Python Virtual Machine (PVM).
- C. Transpiles Python into intermediate JavaScript code to execute inside a runtime engine.
- D. Interprets source code purely line-by-line without any bytecode compilation stage.

Q2. [Basic] Which of the following is an **invalid** variable identifier according to Python syntax rules?

- A. _student_gpa
- B. totalRevenue
- C. 2nd_quarter
- D. MAX_LIMIT

Q3. [Basic] What is the data type returned by the built-in function call `val = input("Enter quantity: ")`?

- A. `int` if the user enters only integer digits, and `float` if a decimal point is present.
- B. Always `str`, regardless of whether the entered characters represent numerical digits.
- C. Automatically inferred as `int`, `float`, or `str` based on content parsing.
- D. `NoneType` until an explicit `typecast` function is called.

Q4. [Basic] What is the exact console output produced by the following two Python statements?

```
print("DSFE", "2026", sep="->", end="#")
print("NEU")
```

- A. DSFE->2026#NEU
- B. DSFE->2026#\nNEU
- C. DSFE 2026->#NEU
- D. DSFE->2026\n#NEU

Q5. [Basic] Which type of error occurs when executing the statement `result = 100 / 0`?

- A. `SyntaxError`, because division by zero is caught by the compiler parser.
- B. `RuntimeError` (`ZeroDivisionError`), raised dynamically during program execution.
- C. `LogicError`, which silently sets `result` to `inf` without terminating.
- D. `TypeError`, because integer division cannot accept zero as an argument.

- Q6. [Medium]** What is the exact numerical result of evaluating `17 // 4 * 2 + 19 % 5 ** 2`?
- A. 27 B. 12 C. 23 D. 4
- Q7. [Basic]** In Python, what is the boolean outcome of `0.1 + 0.2 == 0.3` and why?
- A. True, because Python automatically rounds floating-point values to 2 decimal places.
B. False, because decimal fractions cannot be represented exactly in IEEE 754 binary floating-point representation.
C. True, because the equality operator compares abstract numeric values.
D. A `TypeError` is raised because binary floats cannot be compared directly with literals.
- Q8. [Basic]** Given the string `name = "Python"`, what happens when executing `name[0] = "J"`?
- A. `name` is successfully modified to `"Jython"`.
B. A `TypeError` is raised because strings in Python are immutable objects.
C. A new copy of the string is created while the original `name` remains unchanged.
D. A `ValueError` is raised because index 0 is protected.
- Q9. [Basic]** What is the resulting string evaluated from `"2" * 3 + "4" * 2`?
- A. "14" B. "22244" C. "2342" D. Raises `TypeError`
- Q10. [Basic]** Which formatted string literal (f-string) correctly formats `val = 3.1415926` to 2 decimal places?
- A. `f"{val%2f}"` B. `f"{val:.2f}"` C. `f"{val.round(2)}"` D. `f"{round:val:2}"`
- Q11. [Medium]** Given `text = "DataScience"`, what value is produced by the slice `text[8:1:-2]`?
- A. "niSt" B. "ecni" C. "eicS" D. "nS"
- Q12. [Basic]** What is the fundamental distinction between the operators `=` and `==` in Python?
- A. `=` assigns a value to a variable, whereas `==` tests equality between two operands.
B. `=` tests numeric equality, whereas `==` tests string equality.
C. They are interchangeable within `if` condition headers.
D. `=` performs identity comparison, whereas `==` creates an alias reference.
- Q13. [Basic]** How does Python interpret the chained relational comparison `10 < x <= 25`?
- A. Evaluates `(10 < x) <= 25`, comparing the boolean result of `10 < x` against 25.
B. Equivalent to `10 < x` and `x <= 25`, with `x` evaluated only once.
C. Raises a `SyntaxError` because comparison operators cannot be chained.
D. Equivalent to `10 < x or x <= 25`.

- Q14. [Basic]** What is the precedence hierarchy among boolean operators not, and, and or in Python?
- not (highest), followed by and, followed by or (lowest).
 - and (highest), followed by or, followed by not (lowest).
 - or (highest), followed by and, followed by not (lowest).
 - Equal precedence; strictly evaluated from left to right.
- Q15. [Basic]** Which built-in string method checks whether all characters in string `s` are digits (and non-empty)?
- `s.isnumeric_int()`
 - `s.isdigit()`
 - `s.has_digits()`
 - `s.isnumber()`
- Q16. [Medium]** Applying De Morgan's Laws, which condition is logically equivalent to `not (score >= 50 and status != "probation")`?
- `score < 50 or status == "probation"`
 - `score < 50 and status == "probation"`
 - `score <= 50 or status != "probation"`
 - `not (score >= 50) and not (status != "probation")`
- Q17. [Basic]** Given `score = 82`, what is the final value assigned to `grade` in the following block?
- ```
if score >= 50: grade = "Pass"
elif score >= 80: grade = "Distinction"
else: grade = "Fail"
```
- "Distinction"
  - "Pass"
  - "Pass, Distinction"
  - "Fail"
- Q18. [Basic]** How does Python handle short-circuit evaluation in the expression `A or B`?
- If `A` is True, `B` is not evaluated; the overall expression immediately evaluates to True.
  - If `A` is True, Python still evaluates `B` to verify syntax.
  - If `A` is False, `B` is skipped and False is returned.
  - Both expressions are evaluated concurrently.
- Q19. [Basic]** What is the boolean outcome of the comparison `"banana" > "Apple"` and why?
- True, because lowercase 'b' has a higher Unicode code point (98) than uppercase 'A' (65).
  - False, because uppercase letters precede lowercase letters in reverse ASCII ordering.
  - False, because "Apple" has a shorter length.
  - Raises a TypeError because strings cannot be compared with relational operators.
- Q20. [Medium]** Given `x = 0`, which expression evaluates safely **without** raising ZeroDivisionError?
- `(10 / x > 2) and x != 0`
  - `x != 0 and (10 / x > 2)`
  - `x == 0 or (10 / x > 2)`
  - Both **B** and **C** evaluate safely due to short-circuit evaluation.

## Section B: Coding — Fill-in Problems

(30 points — 2 problems, 10 blanks)

Write missing Python expressions directly into designated answer slots. Difficulty: 24 pts Basic (80%), 6 pts Medium (20%).

### Problem B1: Currency Exchange Teller Utility (12 points — 4 blanks, 3 pts each [All Basic])

Complete the program below by providing expressions for blanks [B1.1] to [B1.4]. The utility converts USD to VND (rate: 25,450 VND/USD), assesses a 2.0% service fee, extracts a terminal code, and prints a receipt.

```
exchange_rate, fee_rate, ref_code = 25450.0, 0.02, "TERM04-TX99281"
[B1.1] Prompt user for USD amount and convert input to float:
usd_amount = _____ # Blank [B1.1] (3 pts - Basic)
base_vnd = usd_amount * exchange_rate
[B1.2] Calculate service fee in VND (base_vnd multiplied by fee_rate):
fee_vnd = _____ # Blank [B1.2] (3 pts - Basic)
total_vnd = base_vnd + fee_vnd
[B1.3] Extract first 6 characters ("TERM04") from ref_code using string slicing:
terminal_id = _____ # Blank [B1.3] (3 pts - Basic)
[B1.4] Print final total formatted with commas and 2 decimals (e.g. 2,595,900.00 VND):
print(f"Total Payable: {_____} VND") # Blank [B1.4] (3 pts - Basic)
```

#### Answer Slots for Problem B1

[B1.1]: ..... [B1.2]: .....  
[B1.3]: ..... [B1.4]: .....

### Problem B2: Bank Loan Risk Tier Classifier (18 points — 6 blanks, 3 pts each)

Difficulty: Blanks [B2.1]–[B2.4] are Basic (12 pts); Blanks [B2.5]–[B2.6] are Medium (6 pts).

Complete the risk assessment engine below by providing expressions for blanks [B2.1] to [B2.6].

```
credit_score = int(input("Enter Credit Score (300-850): "))
monthly_income = float(input("Enter Monthly Income (USD): "))
monthly_debt = float(input("Enter Monthly Debt (USD): "))
has_delinquency = input("Any Delinquency? (yes/no): ").strip().lower() == "yes"
has_guarantor = input("Guarantor available? (yes/no): ").strip().lower() == "yes"
[B2.1] Basic: Validate credit_score is within bounds [300, 850]:
if not (_____): # Blank [B2.1] (3 pts - Basic)
 print("ERROR: Invalid credit score."); exit()
[B2.2] Basic: Healthy Debt-to-Income (DTI) condition (monthly_debt / monthly_income <= 0.40):
is_dti_healthy = _____ # Blank [B2.2] (3 pts - Basic)
[B2.3] Basic: Tier 1 check: score >= 750 AND income >= 3000 AND NO delinquency:
tier1_eligible = _____ # Blank [B2.3] (3 pts - Basic)
card_tier = "DECLINED"
if tier1_eligible and is_dti_healthy:
 card_tier = "PLATINUM PRIME"
[B2.4] Basic: Second tier check: score >= 650 AND income >= 1800:
elif _____: # Blank [B2.4] (3 pts - Basic)
 card_tier = "GOLD STANDARD"
[B2.5] Medium: Short-circuit division guard: income > 0 AND debt ratio < 0.50:
safe_debt_check = _____ # Blank [B2.5] (3 pts - Medium)
[B2.6] Medium: Fallback check: guarantor AND (score >= 600 OR income >= 2500) AND NOT delinquency:
if card_tier == "DECLINED" and safe_debt_check:
 if _____: # Blank [B2.6] (3 pts - Medium)
 card_tier = "BASIC SECURED"
print(f"Decision: {card_tier}")
```

#### Answer Slots for Problem B2

[B2.1] (Basic): ..... [B2.2] (Basic): .....  
 [B2.3] (Basic): ..... [B2.4] (Basic): .....  
 [B2.5] (Medium): ..... [B2.6] (Medium): .....

### Section C: Applied Full-Code Programming & Design

(30 points — 3 problems)

Difficulty: 24 pts Basic (80%), 6 pts Medium (20%). Problems C1 and C2 share the common business scenario below with two distinct requirements. Problem C3 evaluates decision making with branching and string methods.

#### Problem Scenario for C1 & C2: University Banquet Event Dining Invoice Engine

A university banquet dining hall generates an itemized bill for student dinner reservations and calculates equal payment splits among attendees based on the following specifications:

- **Inputs:** Food & beverage subtotal (subtotal in USD, float), dining guests count (num\_diners, int), catering service charge rate (service\_rate, float, e.g. 0.10 for 10%).
- **Financial Formulas:** Service Charge = subtotal × service\_rate; Taxable Subtotal = subtotal + Service Charge; VAT (8%) = Taxable Subtotal × 0.08; Grand Total = Taxable Subtotal + VAT; Per-Diner Share = Grand Total/num\_diners.

#### Problem C1: Algorithmic Specification & Pseudocode (8 points — [All Basic])

Requirement: Formulate structured **pseudocode** only (e.g., using INPUT, COMPUTE / SET, PRINT, BEGIN, END). Do NOT write Python code.

Write a complete algorithm in pseudocode for the Banquet Dining Invoice Engine that reads the inputs, sequentially applies the financial formulas, and outputs the itemized bill and each guest's share.

#### Structured Pseudocode for Problem C1 (8 points — Basic)

### Problem C2: Sequential Python Implementation (10 points — [All Basic])

*Requirement: Write a complete Python program for the banquet engine. Strictly do **NOT** use conditional statements (`if`, `elif`, `else`) or string methods.*

Implement the program following the sequential **Input** → **Process** → **Output** pattern. Prompt for `subtotal`, `num_diners`, and `service_rate`, compute all metrics, and print the formatted receipt (monetary values with 2 decimals and \$):

```
Subtotal: $150.00 | Service (10%): $15.00 | VAT (8%): $13.20
GRAND TOTAL: $178.20 | EACH PERSON PAYS (4 diners): $44.55
```

#### Python Implementation for Problem C2 (10 points — Basic, No If-Then)

### Problem C3: Logistics Freight Shipping & Dimensional Weight Engine (12 points)

*Difficulty Breakdown: 6 points Basic + 6 points Medium. Note: This is the **ONLY** problem requiring string processing methods and conditional branching (if-elif-else).*

An international express courier calculates shipping charges based on tracking code format validation, actual vs. dimensional weight, destination zones, and progressive tariffs. Write a complete Python program according to these specifications:

#### Requirements:

- Input & Validation [4 pts — Basic]:** Prompt for tracking code `pkg_id` (str), weight `weight_kg` (float), dimensions (`length_cm`, `width_cm`, `height_cm`), and destination zone ("DOMESTIC" or "INTERNATIONAL").  
**Validation with String Methods:** `pkg_id` must start with "PKG-" and have length  $\geq 8$ . Weight and all dimensions must be  $> 0$ . Destination zone.`strip().upper()` must be "DOMESTIC" or "INTERNATIONAL". If invalid, display "ERROR: Invalid package parameters." and terminate immediately.
- Dimensional Weight & Progressive Freight [2 pts — Basic]:** Compute volumetric weight: `vol_weight = (length_cm × width_cm × height_cm)/5000`.  
Determine billable weight: `billable_weight = max(weight_kg, vol_weight)`.  
Compute progressive base freight:
  - If `billable_weight`  $\leq 2.0$  kg: Flat charge of **\$12.00**.
  - If  $2.0 < \text{billable\_weight} \leq 10.0$  kg: **\$12.00 + \$3.50/kg** for the portion exceeding 2.0 kg.
  - If `billable_weight`  $> 10.0$  kg: **\$40.00 + \$2.00/kg** for the portion exceeding 10.0 kg.
- International Customs Surcharge [6 pts — Medium]:** If `zone.strip().upper() == "INTERNATIONAL"`, apply a 30% customs surcharge on base freight, subject to a **minimum surcharge of \$25.00** ( $\max(0.30 \times \text{base}, 25.0)$ ). For domestic packages, customs surcharge is \$0.00.  
Total Shipping Cost = base freight + international surcharge.
- Output Format:** Display formatted manifest (weights with 1 decimal, costs with 2 decimals):

```
=====
LOGISTICS FREIGHT DISPATCH MANIFEST
=====
Package ID: PKG-77291 | Billable Weight: 4.8 kg
Base Freight Charge: $21.80 | International Surcharge: $25.00

TOTAL SHIPPING PAYABLE: $46.80
=====
```

#### Python Implementation for Problem C3 (12 points — 6 Basic + 6 Medium)

— END OF EXAMINATION PAPER —